Practical - 14

Practical Title: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

Problem Definition: Develop an admin dashboard that displays user statistics, student count, event count and latest registrations. Restrict access to admin users only and show dynamic menu options based on role.

Theory

1. Are dashboard values loaded dynamically from the database?

Answer: Yes. Dashboard values are loaded dynamically from the MySQL database using PHP and SQL queries. The COUNT() function can calculate the total number of users, students, events, and registrations. Whenever the database records change, the dashboard displays updated statistics when the page reloads and the queries run again.

Reference Link:
https://www.w3schools.com/php/php_mysql_select.asp
https://www.w3schools.com/mysql/func_mysql_count.asp

2. Is access restricted to admin users?
Answer: Yes. Access to the admin dashboard should be restricted to users with the admin role. PHP sessions can store the logged-in user's information and role. Before displaying the dashboard, PHP checks whether the user is logged in and has admin privileges. If the user is not an admin, access is denied or they are redirected to another page

Reference Link:
https://www.w3schools.com/php/php_sessions.asp
https://www.w3schools.com/php/php_if_else.asp

3. Are role-based menus working?

Answer: Yes. Role-based menus display different navigation options according to the logged-in user's role. For example, an admin can access user management, student records, event management, registrations, and audit logs. A normal user can see only the options permitted for their role. PHP conditional statements can be used to display the appropriate menu items.

Reference Link:
https://www.w3schools.com/php/php_if_else.asp
https://www.w3schools.com/php/php_sessions.asp

4. Are important admin actions recorded in audit_logs?

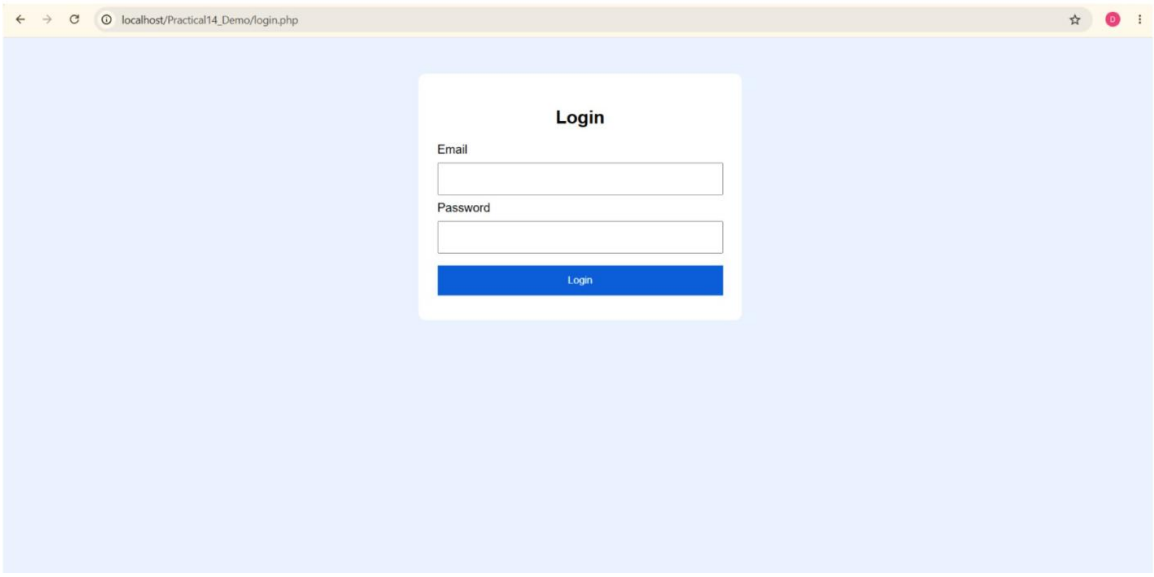
Answer: Yes. Important administrative activities, such as adding or deleting users, updating student records, changing user status, and modifying events, should be recorded in the audit_logs table. Each log entry can contain the admin's user ID, the action performed, the affected record, and the date and time. SQL INSERT statements can store these details, helping administrators track activities and investigate suspicious changes.

Reference Link:
https://www.w3schools.com/php/php_mysql_insert.asp
https://www.w3schools.com/mysql/mysql_insert.asp

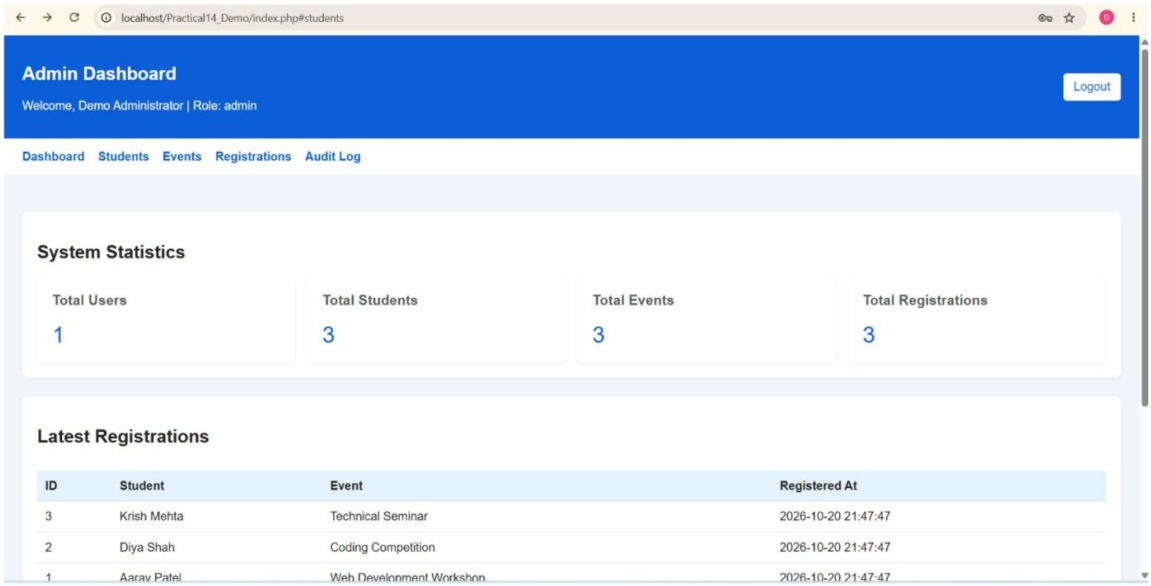
A. Implementation/ Output/Result

```
B.
C. <?php
D. mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
E.
F. try {
G.     $conn = new mysqli(
H.         "localhost",
I.         "root",
J.         "",
K.         "student_management"
L.     );
M.
N.     $conn->set_charset("utf8mb4");
O. } catch (mysqli_sql_exception $e) {
P.     error_log($e->getMessage());
Q.     http_response_code(500);
R.     exit("Database connection failed.");
S. }
T. ?>
U.
```

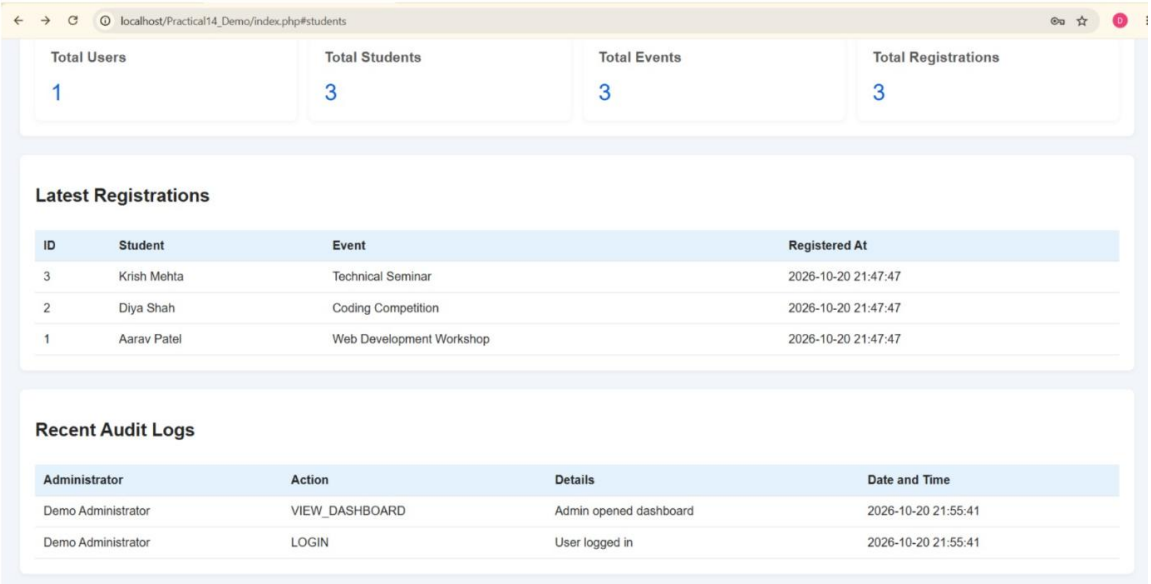
B. Evidence of Work (Screenshots)



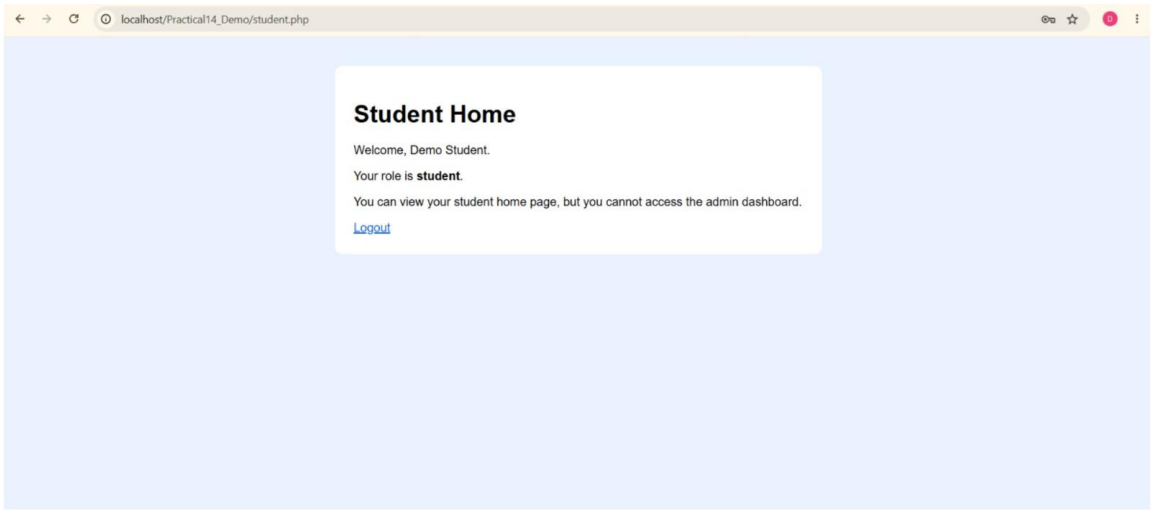
Description: This screenshot shows the login page of the Student Management Admin Dashboard developed using PHP and MySQL. The page contains email and password fields along with a Login button for user authentication. It provides an entry point to the system, where the administrator must enter valid login credentials to access the dashboard. This helps implement secure access and supports the Role-Based Access Control (RBAC) requirement of Practical 14 by restricting access to authorized users



Description: This screenshot shows the Admin Dashboard after successful login as a Demo Administrator with the admin role. The dashboard displays system statistics, including the total number of users (1), students (3), events (3), and registrations (3). It also shows a Latest Registrations table containing registration IDs, student names, event names, and registration timestamps. The navigation menu provides access to Dashboard, Students, Events, Registrations, and Audit Log sections, while the Logout button allows the administrator to end the session. The screenshot demonstrates the use of dynamic database queries to display statistics and registration records, role-based dashboard access, and the interface for monitoring administrative activities.



Description: The admin dashboard displays the total number of users, students, events, and registrations retrieved from the database. It includes a Latest Registrations table showing student names, event names, and registration dates. The Recent Audit Logs section records administrative activities, such as login and dashboard access, helping monitor system usage and maintain activity traceability.



Description: This screenshot demonstrates the role-based access control (RBAC) feature of the Student Management System. It shows a logged-in user with the student role accessing the Student Home page, while access to the admin dashboard is restricted. The page also provides a Logout option, demonstrating basic user session management and role-based access restriction as required in this Practical

C. GitHub Repository Link:

https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical14_Demo

D. Challenges Faced:

1. Database connection: Connecting PHP to MySQL and ensuring the dashboard retrieves the correct data from all required tables.

2. Dynamic statistics: Writing SQL queries to calculate the total number of users, students, events, and registrations accurately.

3. Role-based access control: Restricting the admin dashboard to authorized users and preventing normal users from accessing protected pages.

4. Dynamic menu creation: Displaying the correct menu options according to the logged-in user's role.

5. Audit log management: Recording important admin activities in the audit_logs table with the correct user ID, action, and timestamp.

6. Dashboard design: Creating a clear, responsive, and user-friendly interface using Bootstrap.

7. Testing and debugging: Checking database queries, login permissions, menu visibility, and audit log entries to ensure everything works correctly.

E. Learning Outcomes:

- Develop an admin dashboard using PHP, MySQL, and Bootstrap.
- Retrieve and display dynamic statistics from database tables using SQL queries.
- Implement role-based access control (RBAC) to restrict access to authorized administrators.
- Create dynamic navigation menus based on user roles.
- Record important administrative activities in an `audit_logs` table for tracking and accountability.
- Connect PHP applications to MySQL databases and perform database operations.
- Test and debug dashboard functionality, access permissions, and audit log records.
- Understand the importance of security, monitoring, and traceability in web applications.

Faculty Remarks: (Keep this blank for faculty)